

Multi-Agent Learning and Imperfect Information Games

Multi-Agent Learning and Imperfect Information Games

Public Edition of the Final Report

Author: Mauricio Garcia Villanueva

Project: Liar's Dice CFR Lab

Live demo: emgarviii.github.io/liars_dice/

Code: github.com/emgarVIII/liars_dice

Executive Summary

This project started as a university research project about multi-agent learning, imperfect-information games, and computational game solving. I wanted to understand how ideas from poker AI, especially Counterfactual Regret Minimization, could apply to a game that felt more personal and easier to explain than a full poker environment. I chose Liar's Dice because it has the properties I cared about: private information, strategic claims, bluffing, and adversarial decision making.

The public version of the project is intentionally honest about scope. It is not a full solver for classic Liar's Dice. Instead, it is a simplified one-claim challenge abstraction. One player makes a claim about the total dice on the table, and the other player chooses whether to believe or challenge that claim. That smaller game still captures the central learning problem: each player must act with incomplete state information, because their opponent's dice are hidden.

The final artifact is an end-to-end research engineering demo. Python code trains a CFR-style policy offline, exports normalized policy JSON and metrics JSON, and a TypeScript website loads those artifacts as a static GitHub Pages demo. The result is a playable interface, a benchmark suite, best-response-style diagnostics, and a paper archive.

The main lesson was not that the AI is solved or perfect. The strongest lesson was that evaluation matters. A count-aware policy improved benchmark average win rate from 45.3 percent to 73.0 percent across seeded opponent profiles, but best-response pressure still showed exploitable behavior. That tension is the core story of the project: train the policy, measure the improvement, then state the limitation instead of hiding it.

Research Question

The question I kept returning to was:

Can a simplified imperfect-information dice game be modeled, trained, evaluated, and published as a reproducible AI/ML portfolio artifact?

That question has two parts. The first is technical: how do I represent hidden information, train policies through self-play, and evaluate the resulting strategy? The second is communicative: how do I turn a

course project into something another engineer, recruiter, or technical interviewer can understand without reading every implementation detail?

The final website answers both parts. It exposes the game mechanics through play, shows the pipeline behind the policy, and explains why the results are useful without claiming more than the evidence supports.

Why Liar's Dice

Liar's Dice is a natural imperfect-information game. Each player knows their own dice but not the opponent's dice. A claim can be true, false, or uncertain depending on the hidden hand. Good play requires probability, bluffing, skepticism, and response selection under uncertainty.

That made it a useful bridge between classroom game theory and applied AI. The same kind of reasoning appears in domains I care about:

- hidden state in markets and risk systems,
- adversarial behavior from other agents,
- probabilistic decision policies,
- stress testing a strategy against stronger opponents,
- separating benchmark success from robust play.

The full classic game also has a raise loop. Players keep increasing bids until someone challenges. That version is important, and the website now includes a playable classic comparison mode. But the trained policy does not claim to solve that mode. The trained-and-evaluated public demo focuses on the simplified one-claim game because that abstraction keeps the learning problem tractable and explainable.

Game Abstraction

The public research mode has three steps.

Both players roll private dice.

The claimant makes one claim of the form `claim_Q_F`, meaning at least quantity `Q` dice across the whole table show face `F`.

The responder chooses believe or challenge.

If the claim is true and the responder believes it, the claimant succeeds. If the claim is false and the responder challenges it, the responder succeeds. The loser gives up a die, and the next round begins.

This abstraction removes the bid ladder, repeated raising, multiplayer turn order, and wild-face variants. That tradeoff is deliberate. It keeps the part I wanted to study, hidden information and adversarial response selection, while removing enough complexity to make the project reproducible on a local machine.

The website labels this clearly because it is easy to confuse the demo with classic Liar's Dice. The classic route exists for comparison, but it uses a heuristic AI and is not the source of the CFR-style policy or benchmark claims.

Information Sets and Policies

The key game-solving idea is the information set. A player does not know the full state of the game. Instead, the player knows only the information available at decision time.

In this project, an information set includes:

- the player's private dice,
- the public remaining dice counts,
- the public claim, when responding,
- the legal actions available from that state.

The policy is a probability distribution over legal actions for each information set. The claimant policy assigns probabilities to possible claims. The responder policy assigns probabilities to believe and challenge.

An early baseline used a weaker policy key that was too dependent on the private hand alone. The published policy is count-aware. It conditions on both public dice counts and private dice, which matters because a claim that is reasonable with ten total dice can be impossible or reckless later in the match.

Training Pipeline

The training code is offline Python. The browser does not train live. This separation is important because it keeps the public site static and reproducible.

The pipeline is:

- Define the simplified rules and legal actions.

- Generate sampled game states.

- Train CFR-style claim and response policies through self-play.

- Normalize and export the policy as JSON.

- Run deterministic benchmark simulations.

- Export metrics as JSON.

- Load those frozen artifacts in the TypeScript website.

CFR-style training is useful here because it updates strategies by tracking regret. In simple terms, the algorithm repeatedly asks: if I had chosen another legal action in this information set, would I have done better? Actions with positive regret receive more probability over time. CFR+ style regret clipping helps stabilize learning by keeping negative regret from dragging future updates.

I use the phrase CFR-style carefully. The current project is an applied, simplified implementation around a tractable abstraction. It is inspired by the game-solving methods used in larger imperfect-information systems, but the current artifact does not prove a full Nash equilibrium for Liar's Dice.

Evaluation

The most important public improvement was adding evaluation instead of only showing a policy. The selected count-aware policy was trained for 200,000 sampled CFR+ style iterations and compared against the earlier baseline.

The benchmark suite includes:

- random claims and random responses,
- random claims and skeptical responses,
- random claims and threshold responses,
- truth-biased claims and threshold responses.

Across these seeded opponent profiles, the baseline averaged 45.3 percent AI win rate. The selected policy averaged 73.0 percent. That is a meaningful improvement, especially because the policy no longer reuses a fixed five-dice strategy as dice counts change.

But the stronger diagnostic is the best-response-style check. That test asks what a more targeted opponent can find against the policy. The result was mixed. The responder side was closer to robust, but the claimant side remained exploitable. The published best-response pressure is 0.491, which is a warning rather than a trophy.

This is why the results page says “stronger, not solved.” The project improved benchmark play and also measured a weakness. For an engineering portfolio, that is more credible than claiming equilibrium without enough evidence.

What the Project Demonstrates

The project demonstrates that I can take a research idea and turn it into a working artifact:

- define a canonical rule model,
- implement policy training,
- export machine-readable artifacts,
- evaluate with deterministic benchmarks,
- build a playable static website,
- preserve research documents,
- explain limitations clearly.

It also demonstrates a practical lesson about AI systems. A model can look strong against fixed benchmarks while still having exploitable structure. That is exactly why adversarial diagnostics, stress tests, and careful public claims matter.

What the Project Does Not Prove

The project does not prove that the policy is a Nash equilibrium.

It does not solve full classic Liar's Dice.

It does not show that CFR-style learning alone is enough to exploit every weak opponent.

It does not remove the need for stronger exploitability minimization, broader game trees, or more formal convergence analysis.

Those limits are not side notes. They are part of the result. The public version is more valuable because it is explicit about them.

Lessons Learned

The first lesson was that abstraction is not a shortcut around rigor. Simplifying the game made the work possible, but it also created a responsibility to label the abstraction clearly. If the site did not explain that distinction, visitors could assume the project solved the original game.

The second lesson was that a policy is only as persuasive as its evaluation. A high win rate is useful, but only when the opponent profiles are named and the failure modes are also shown.

The third lesson was that research work needs a public interface. The original course project had ideas, code, and papers, but it was not fully displayable. Turning it into a website forced the project to become clearer, more reproducible, and more honest.

Future Work

The next technical step is to reduce exploitability directly. That could mean training against stronger best-response pressure, improving the claim policy, or adding a more formal exploitability estimate.

The next game-design step is to extend the solver toward classic raise-and-challenge Liar's Dice. That would require modeling the bid ladder, repeated action histories, and larger information sets.

The next presentation step is to keep the public artifact easy to scan. A recruiter should understand the project in 30 seconds. A technical interviewer should be able to dig into the method, results, and code.

Closing Reflection

This project matters to me because it connects several areas I care about: AI, game theory, multi-agent learning, and decision making under uncertainty. It is imperfect, but it is real. It has code, tests, metrics, a playable interface, original papers, and measured limitations.

That is the version of the project I want to show publicly: not a claim of perfect play, but evidence that I can take an ambiguous research idea and turn it into a validated engineering artifact.